

Emergencies and Utility Service Faults and Damages Report

CHAPTER 3: RESEARCH METHODOLOGY AND PROPOSED SYSTEM

3.1 Introduction

This chapter presents the methodology adopted for the design and development of the Emergency and Utility Service Fault Reporting Mobile Application for residents of the Eastern Region of Ghana. The chapter explains the software development process model used for the project, the justification for selecting the model, and the activities carried out in each phase of the development process.

The chapter further presents the proposed system, including the system models, features, development tools, and development environment. UML diagrams such as the context diagram, sequence diagram, structural model, and behavioral model are used to describe the architecture and interactions within the system. In addition, the chapter discusses the strengths and possible limitations of the proposed system.

3.2 Software Development Methodology

3.2.1 Introduction to Agile Methodology

The Agile software development methodology was adopted for the development of this project. Agile is an iterative and incremental software development approach that focuses on flexibility, collaboration, continuous improvement, and rapid delivery of functional software components. Agile encourages frequent interaction with stakeholders and allows developers to respond quickly to changing requirements throughout the development process.

Unlike traditional plan-driven methodologies such as the Waterfall model, Agile supports continuous testing, regular feedback, and iterative development through short development cycles known as sprints. This makes Agile suitable for projects where requirements may evolve during development.

3.2.2 Justification for Choosing Agile Methodology

The Agile methodology was selected for this project because of its flexibility and ability to support continuous improvement during development. The Emergency and Utility Service Fault Reporting Mobile Application required regular adjustments and feature enhancements throughout the development process, making Agile more appropriate than rigid plan-driven models.

The following reasons justify the use of Agile methodology for this project:

1. Flexibility in Requirement Changes

The project requirements evolved during development as new ideas and user feedback emerged. Agile supports changes at any stage of development without affecting the overall progress of the project.

2. Iterative Development

The project was developed in small phases known as sprints. Each sprint delivered functional features such as user authentication, report submission, GPS integration, multimedia uploads, and notifications.

3. Continuous Testing and Feedback

Agile allowed testing to be performed continuously after each sprint. This helped identify errors early and improved system reliability.

4. Improved User Involvement

The methodology encouraged regular feedback from potential users and stakeholders, ensuring that the developed system met user expectations and requirements.

5. Faster Delivery of Functional Components

Agile enabled the development of working features incrementally, allowing portions of the system to become operational before the entire project was completed.

6. Risk Reduction

By dividing the project into smaller manageable phases, risks associated with system failure or major design problems were minimized.

3.2.3 Agile Software Development Process

The Agile development process used in this project consisted of the following phases:

1. Requirement Gathering and Planning
2. System Design
3. Development and Implementation
4. Testing and Debugging
5. Deployment
6. Maintenance and Improvement

The development activities were organized into iterative sprints, with each sprint focusing on specific features of the system.

3.3 Activities Performed in Each Agile Phase

3.3.1 Requirement Gathering and Planning

This phase involved identifying the functional and non-functional requirements of the proposed system. Information was gathered through literature review, observation of existing emergency reporting systems, and analysis of the challenges associated with emergency and utility service reporting in Ghana.

The following activities were carried out:

1. Identification of system users and stakeholders.
2. Definition of project objectives and scope.
3. Collection of functional requirements such as:
 1. User registration and login.
 2. Emergency and utility fault reporting.
 3. GPS location tracking.
 4. Multimedia attachments.
 5. Notifications and report tracking.
4. Collection of non-functional requirements such as:
 1. Security.
 2. Reliability.
 3. Usability.
 4. Performance.
5. Planning of project timeline and sprint activities.

3.3.2 System Design

The system design phase focused on creating the architecture and models of the proposed system.

The following activities were carried out:

1. Designing the database schema using MySQL.
2. Designing the mobile application user interface.
3. Creating API structure for communication between frontend and backend.
4. Designing UML diagrams including:
5. Designing authentication and notification workflows.
6. Designing GPS and multimedia integration processes.

3.3.3 Development and Implementation

This phase involved the actual coding and implementation of the system components.

The following technologies were used:

1. React Native for the mobile frontend.
2. Express.js for backend API development.
3. MySQL for database management.
4. Ngrok for API tunneling and testing.
5. Postman for API testing.
6. WebStorm IDE for development.

The following modules were implemented:

1. User Authentication Module.
2. Emergency Reporting Module.
3. Utility Fault Reporting Module.
4. GPS Location Tracking Module.
5. Multimedia Upload Module.
6. Notification System.
7. Admin and Monitoring Module.

3.3.4 Testing and Debugging

Testing was performed continuously throughout development to ensure that all features worked correctly.

The following testing methods were used:

Unit Testing

Each module was tested independently to verify correct functionality.

Integration Testing

Communication between frontend, backend APIs, and database components was tested.

System Testing

The complete system was tested to ensure that all modules worked together correctly.

User Acceptance Testing

Potential users tested the system to evaluate usability and functionality.

The following tools were used during testing:

1. Postman for API testing.
2. Android Emulator and physical mobile devices.
3. MySQL Workbench for database verification.

3.3.5 Deployment

After successful testing, the application was deployed for demonstration and evaluation.

The following activities were carried out:

1. Backend server setup.
2. Database deployment.
3. Mobile application packaging.
4. API exposure using Ngrok.
5. Final system configuration.

3.3.6 Maintenance and Improvement

Maintenance activities involved fixing bugs, improving performance, and preparing recommendations for future enhancements.

Possible future improvements include:

1. Offline emergency reporting.
2. Integration with national emergency services.
3. Real-time dispatcher tracking.
4. AI-based emergency prioritization.
5. Expansion to other regions of Ghana.

3.4 Proposed System

3.4.1 Introduction of the Proposed System

The proposed system is a mobile-based Emergency and Utility Service Fault Reporting System designed for residents of the Eastern Region of Ghana. The system enables users to report emergencies and utility service faults quickly using a mobile application.

The application integrates GPS technology to automatically capture the user's location and transmit accurate location data to service providers. Users can also attach photos, videos, or documents to support their reports.

The system is designed to improve communication between residents and emergency or utility service providers by reducing delays associated with traditional reporting methods such as phone calls and physical visits.

3.5 Models of the Proposed System

3.5.1 Context Model

The context model illustrates the interaction between the proposed system and external entities.

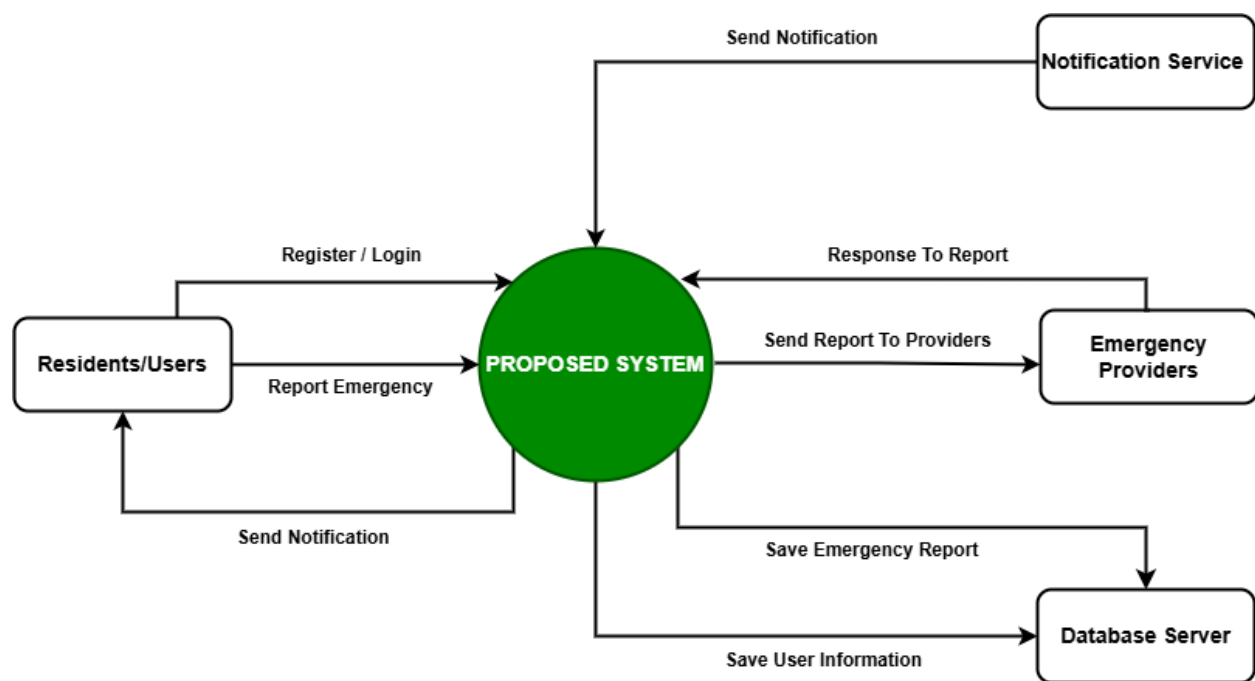


Figure 3.1

Description

The context model for the system consists of several external entities that interact with the main platform to support its operations and services. Emergency Service Providers are responsible for handling emergency requests and responding to incidents reported through the system, while Utility Service Providers provide utility-related services such as electricity, water, and waste management updates. GPS and Map Services supply real-time location tracking and mapping functionalities to help users and responders identify incident locations accurately. Notification Services enable the delivery of alerts, SMS, emails, and push notifications to users and service providers. The Database Server stores, manages, and retrieves all application data securely, including user information, reports, and service records. Finally, Residents or Users interact

directly with the system to report issues, request assistance, receive notifications, and access available services, making them the primary actors within the context model.

3.5.2 Interaction Model (Sequence Diagram)

The sequence diagram describes how the user interacts with the system during report submission.

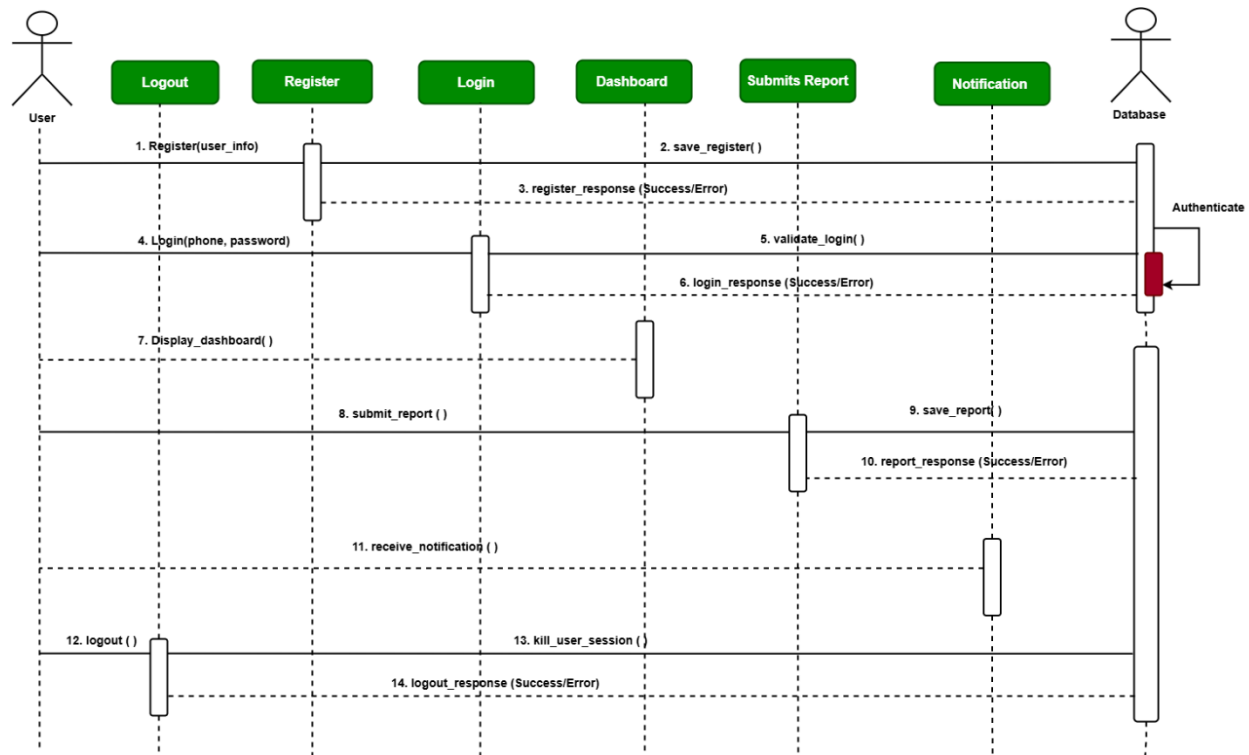


Figure 3.2

Description

The sequence flow of the system begins when the user opens the mobile application and logs into the system to access its services. After successful authentication, the user selects either an emergency report or a utility report type and proceeds to enter the necessary incident details. The system then automatically captures the user's GPS location to provide accurate positioning information for responders. The user may also upload multimedia files such as images or videos to support the report before submitting it to the system. Once submitted, the backend server validates the provided data to ensure completeness and accuracy. After validation, the report is stored securely in the MySQL database for record management and future reference. The system then sends notifications to the relevant service providers responsible for handling the reported issue. Finally, the user receives a confirmation notification indicating that the report has been successfully submitted and processed.

3.5.3 Structural Model (Class Diagram)

The structural model describes the main classes of the proposed system.

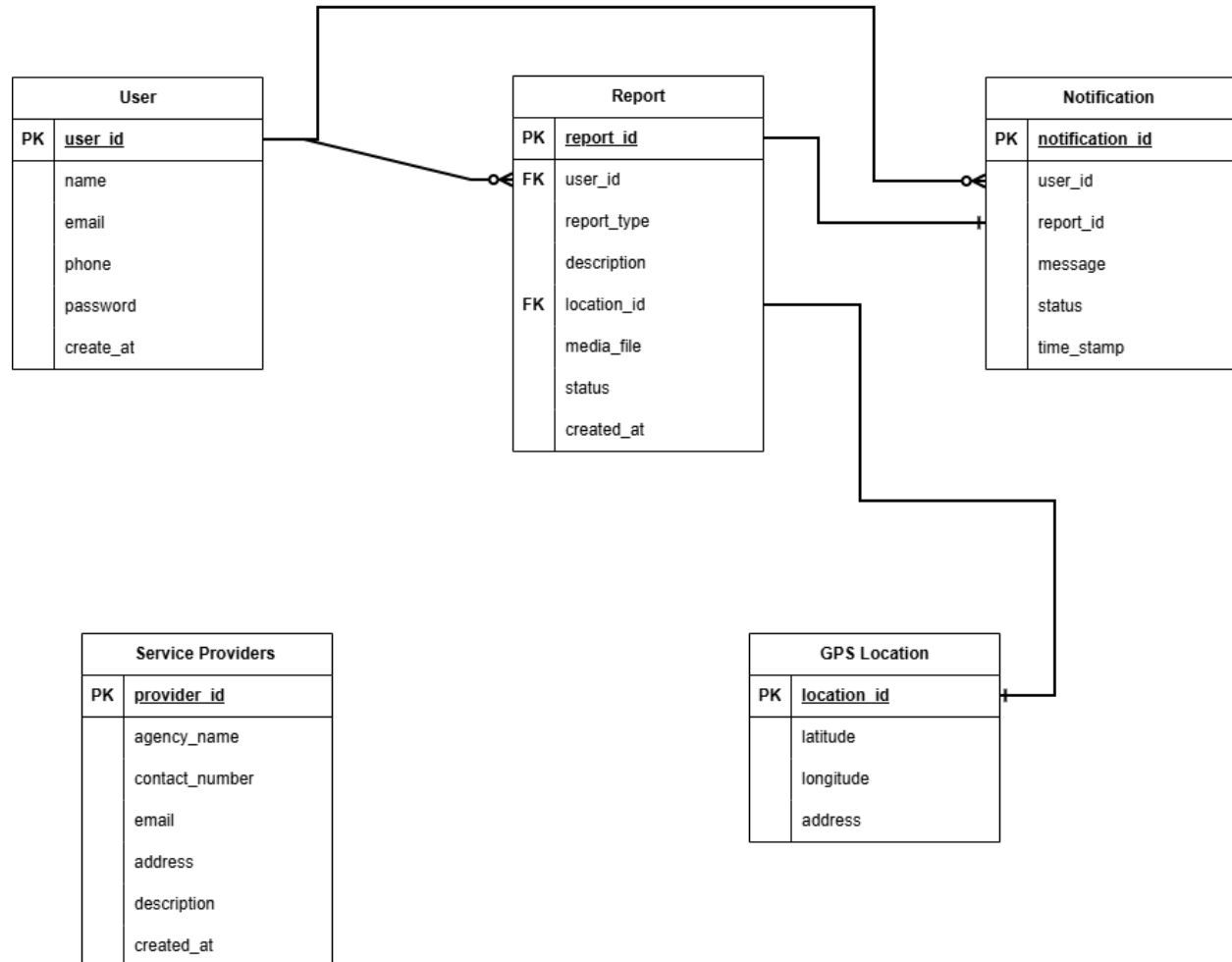


Figure 3.3

Description

The class structure of the system is composed of several main classes that work together to support emergency and utility reporting functionalities. The User class stores user-related information such as UserID, Name, Email, Phone, and Password, enabling authentication and user management within the system. The Report class manages all incident reports submitted by users and contains attributes including ReportID, ReportType, Description, Location, MediaFile, Status, and DateCreated to track and process reports effectively. The Notification class handles communication between the system and users by storing NotificationID, Message, Status, and Timestamp details for alerts and confirmations. The ServiceProvider class represents organizations responsible for responding to reports, with attributes such as ProviderID, AgencyName,

ContactNumber, and Email for identification and communication purposes. Lastly, the GPSLocation class stores geographical information including Latitude, Longitude, and Address to provide accurate location tracking for reported incidents and improve response efficiency.

3.5.4 Behavioral Model (Use Case Diagram)

The use case diagram illustrates how users interact with the Emergency Reporting System and how all activities are connected to the central database for storage, validation, and retrieval of information.

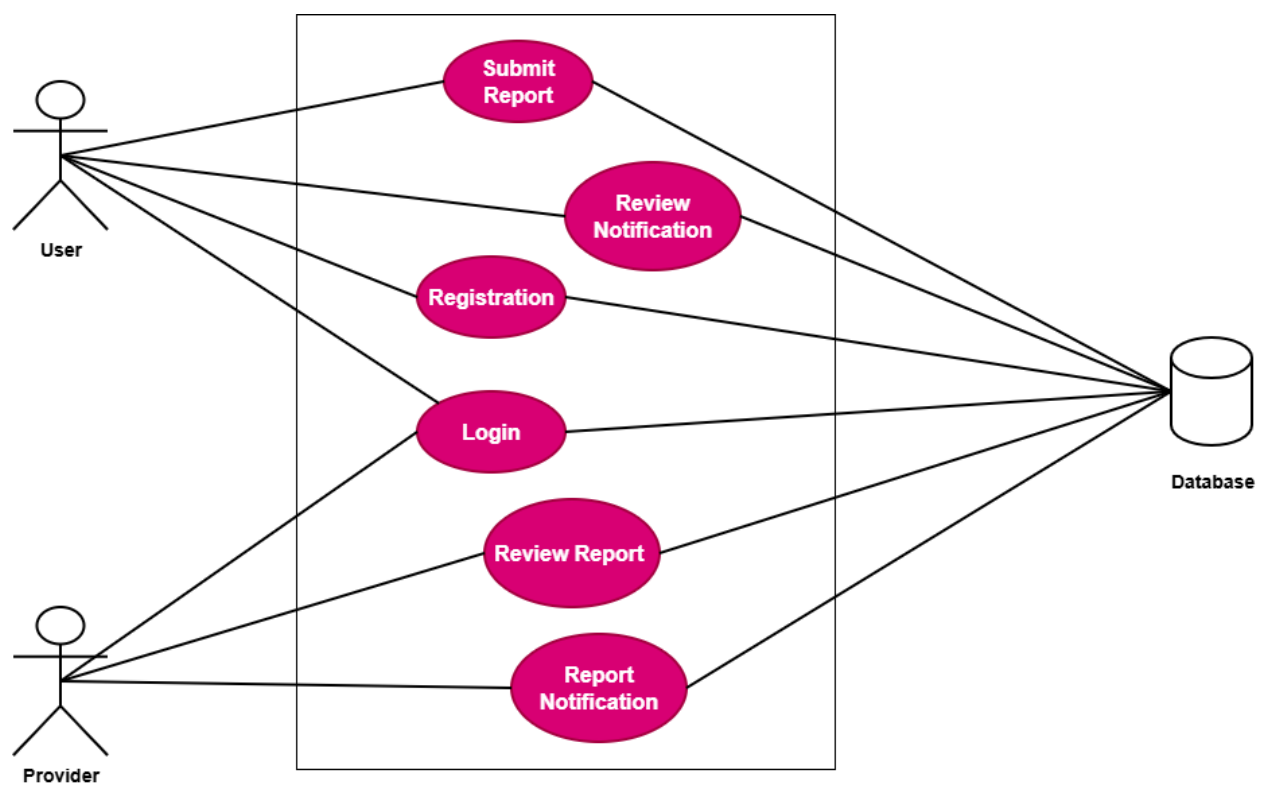


Figure 3.4

Description

The behavioral model of the Emergency Reporting System is represented using a use case approach to show the interactions between users, service providers, and the system database. The process begins when the user authenticates their account through the mobile application. After successful authentication, the user selects a report category and enters incident details. The system automatically captures GPS coordinates and allows the attachment of multimedia files such as images or videos. Once the report is submitted, the system validates the information and stores all related data in the database. Notifications are then sent to the appropriate service provider, who

reviews the report and initiates the required response action. Throughout the process, all activities, updates, and communications are connected to and managed through the central database. Finally, feedback regarding the report status is sent back to the user.

3.6 Features of the Proposed System

The proposed system includes the following features:

1. User registration and authentication.
2. Emergency incident reporting.
3. Utility service fault reporting.
4. GPS-based location tracking.
5. Multimedia file attachments.
6. Automated notifications.
7. Real-time report submission.
8. Secure backend APIs.
9. User-friendly mobile interface.
10. Incident monitoring and tracking.
11. Database storage and retrieval.
12. Confirmation and feedback system.

3.7 Development Tools and Development Environment

The following development tools and technologies were used in the implementation of the proposed system.

3.7.1 React Native

React Native was used for developing the mobile application frontend. It supports cross-platform mobile development and provides a responsive and interactive user interface.

Advantages

1. Cross-platform compatibility.
2. Fast development process.
3. Reusable components.
4. Large community support.

3.7.2 Express.js

Express.js was used for backend API development. It handled routing, request processing, authentication, and communication between the frontend and database.

Advantages

1. Lightweight framework.
2. Fast API development.
3. Easy integration with databases.
4. Scalable architecture.

3.7.3 MySQL Database

MySQL was used as the database management system for storing user data, reports, notifications, and service provider information.

Advantages

1. Reliable data storage.
2. Structured data management.
3. High performance.
4. Secure database operations.

3.7.4 Postman

Postman was used for API testing and debugging.

Uses

1. Testing API endpoints.
2. Verifying request and response data.
3. Debugging backend services.
4. Monitoring API performance.

3.7.5 Ngrok

Ngrok was used to expose local backend APIs to the internet during development and testing.

Uses

1. API tunneling.
2. Remote testing.
3. Mobile application integration.

3.7.6 WebStorm IDE

WebStorm IDE was used as the primary development environment for coding and managing the project.

Advantages

1. Intelligent code completion.
2. Debugging tools.

3. Integrated terminal.
4. Git integration.
5. Improved developer productivity.

3.8 Review of the Good Features of the Proposed System

The proposed system offers several advantages that improve emergency and utility fault reporting.

1. Accurate GPS location tracking improves response efficiency.
2. Multimedia attachments provide additional evidence and context.
3. Real-time notifications improve communication.
4. Mobile accessibility allows users to report incidents quickly.
5. Structured reporting reduces misinformation.
6. Centralized database improves data management.
7. Agile development ensured continuous improvement and testing.
8. Cross-platform mobile support increases accessibility.

3.9 Review of the Bad Features of the Proposed System

Despite its advantages, the proposed system has some limitations.

1. Dependence on internet connectivity.
2. Requirement for smartphones.
3. Possible GPS inaccuracies in remote areas.
4. Dependence on third-party services such as maps and cloud infrastructure.
5. Limited functionality in areas with poor mobile network coverage.
6. Potential privacy concerns related to location tracking.

These limitations may affect system accessibility and performance in certain rural communities.

3.10 Summary of the Chapter

This chapter presented the methodology and proposed system for the Emergency and Utility Service Fault Reporting Mobile Application. The Agile software development methodology was selected due to its flexibility, iterative nature, and ability to support continuous testing and feedback.

The chapter discussed the phases of the Agile process and the activities performed during each phase of development. It also presented the proposed system architecture, UML models, system features, development tools, and development environment.

The strengths and limitations of the proposed system were also analyzed. Overall, the proposed system is designed to improve emergency and utility service reporting through accurate geolocation tracking, multimedia reporting, and real-time communication between users and service providers.